MQIL — Project Blueprint

Model Checking based Quantification of Information Leakage

Presentation-ready explanation of the architecture and workflow described in the paper.

1. Core Idea

MQIL creates **two copies of the same program**, gives the copies different sensitive inputs but the same public inputs, adds assumptions and assertions, and then uses CBMC to check whether corresponding variables differ. Failed assertions indicate information flow from the sensitive input.

Original C Program
↓
Create two copies P1(H1, L, V1, O1) P2(H2, L, V2, O2)
↓
Assumption H1 ≠ H2 **Same public input** L is shared
↓
Add assertions assert(V1 == V2) assert(O1 == O2)
↓
MITER PROGRAM
↓
CBMC
↓
Failed assertions → Information Leakage

2. What Goes Into the Miter?

The paper models a program as **P(H, L, V, O)**:

Symbol	Meaning
H	High / sensitive inputs, such as a secret key
L	Low / non-sensitive inputs
V	Program variables
O	Output variables

3. Why Two Copies?

The two executions are compared while changing only the sensitive input. If a variable changes when the secret changes, the variable is considered to have information flow from the secret.

```
secret1 = 5    secret2 = 8
x = 10 in both copies

a1 = secret1 + x = 15
a2 = secret2 + x = 18

assert(a1 == a2) → FAILED → a is leaky
```

4. What Does CBMC Produce?

CBMC result	Interpretation
Assertion passes	No leakage detected through that checked variable
Assertion fails	The sensitive input influences that variable/output
Count of failed assertions	Total leakage measure used by MQIL

5. Complete MQIL Implementation Blueprint

Step	Module	What happens?
1	**C Program**	Source program containing sensitive and public inputs.
2	**C Parser**	Python + pycparser obtain the program's AST.
3	**AST Modification**	Generate two copies of the program.
4	**Assumption**	Set corresponding sensitive inputs to different values: H1 ≠ H2.
5	**Assertions**	Compare corresponding variables and outputs: V1 == V2 / O1 == O2.
6	**Miter Generation**	Combine both copies, assumptions and assertions into one C program.
7	**CBMC**	Run bounded model checking on the generated miter.
8	**Leak Quantification**	Count failed assertions and identify leaky variables.
9	**Relative Security**	Compare leakage of source and optimized programs.

6. Compiler-Optimization Experiment

The authors also compare source and optimized versions to investigate whether LLVM optimization introduces additional leakage.

Source path	Optimized path
Original C (S)	Original C → LLVM -O3 → optimized LLVM IR → llvm2c → Optimized C (Op)
MQIL → source leakage	MQIL → optimized leakage
■	
Compare leakage → Relative Security (RS)	

7. What the Paper Solves vs. What It Does Not

They implemented	They did not implement
Detect information leakage	Automatically prevent/fix the leakage
Quantify leakage	Automatic repair of LLVM optimizations
Handle explicit and implicit flows	A complete post-fixing mechanism
Analyze loop leakage for bounded loops	General solution for unbounded loops
Compare source vs optimized security	Automatic root-cause repair pipeline

8. Final Presentation Slide

One-line explanation: MQIL takes a C program, creates two copies with different secret inputs but the same public inputs, adds comparison assertions, and uses CBMC to identify and quantify information leakage from failed assertions.

Presentation Script

“The key idea of MQIL is the miter program. We take the original C program and generate two copies. Both copies receive the same public inputs, but their sensitive inputs are different. We then compare corresponding variables and outputs using assertions. If changing only the secret causes an assertion to fail, that variable is considered leaky. CBMC performs the verification, and the number of failed assertions is used to quantify the leakage. The paper then applies this approach to source and LLVM-optimized cryptographic programs.”

Key takeaway: The paper successfully provides a mechanism to *detect and quantify* information leakage; automatic prevention/fixing is left as future work.